

TP noté : API de réservation de salles



Django REST Framework — 2 heures, en groupe

Stephane Fedim

Master 2 Data Science — Développement Web en Python

Énoncé et barème

Table des matières

1	Contexte et objectif	2
2	Modalités	2
3	Démarrage	2
3.1	Ce qui est fourni, ce qui est à faire	3
4	Les données	3
5	Travail demandé	3
5.1	Tâche 1 — Serializers	3
5.2	Tâche 2 — Validation métier	3
5.3	Tâche 3 — ViewSets et router	4
5.4	Tâche 4 — Permissions	4
5.5	Tâche 5 — Action d'occupation	4
5.6	Bonus	4
6	Barème (sur 20)	5
7	Conseils	6
8	Livrables	6

1 Contexte et objectif

Un établissement veut exposer une **API REST** pour gérer ses salles (amphithéâtres, salles de TD, laboratoires, salles de réunion) et leurs réservations. Les modèles et des données de test vous sont fournis : votre travail porte sur l'**API** elle-même.

Ce que ce TP évalue

Serializers et validation métier, ViewSets et routers, permissions personnalisées, action personnalisée — c'est-à-dire ce que nous avons vu en Django REST Framework. La difficulté du sujet est concentrée sur une règle métier : **une salle ne peut pas être réservée deux fois sur le même créneau**.

2 Modalités

- **Durée** : 2 heures, **en groupe de 3**
- **Dépôt GitHub** : un membre du groupe crée le dépôt à partir du squelette fourni, ajoute les deux autres comme collaborateurs, et le partage avec l'enseignant
- **Chaque membre commit sous son propre compte**, régulièrement, avec des messages de commit explicites
- **Rendu** : seuls les commits poussés avant la fin du créneau sont pris en compte

Collaboration

L'historique Git fait partie de la note (cf. barème). Un déséquilibre flagrant et non justifié entre les membres pourra être pris en compte dans l'évaluation individuelle.

3 Démarrage

Récupérez le squelette du projet, placez-le dans votre dépôt, puis :

```
python -m venv .venv
source .venv/bin/activate          # Windows : .venv\Scripts\activate
pip install -r requirements.txt
python manage.py migrate
python manage.py seed
python manage.py runserver
```

L'API est alors sur <http://127.0.0.1:8000/api/>. Le login de la Browsable API est sur [/api-auth/login/](http://127.0.0.1:8000/api-auth/login/).

Compte	Mot de passe	Rôle
alice, bob, charlie	motdepasse123	utilisateurs ordinaires
admin	admin123	staff / super-utilisateur

3.1 Ce qui est fourni, ce qui est à faire

Fourni	À écrire par vous
models.py et migrations	salles/serializers.py
admin.py	salles/views.py
seed (données de test)	salles/permissions.py
settings.py (DRF configuré)	salles/urls.py
config/urls.py	README.md (à compléter)

4 Les données

Modèle	Champs
Salle	nom (unique), capacite, batiment
Reservation	salle (FK), utilisateur (FK), debut, fin, motif, statut (CONFIRMEE ou ANNULEE, par défaut CONFIRMEE), cree_le

Les données de test se situent dans la **semaine du lundi 2 novembre 2026** (heures en UTC). Par exemple, l'*Amphi A* est réservé le lundi de 8h à 10h (alice) et de 14h à 16h (bob), le mardi de 9h à 12h (alice), et une réservation du mercredi (8h à 10h) est ANNULEE.

5 Travail demandé

Routes attendues (le préfixe `/api/` est déjà fourni par `config/urls.py`) :

Route	Rôle
<code>/api/salles/</code> et <code>/api/salles/{id}/</code>	CRUD des salles
<code>/api/reservations/</code> et <code>/api/reservations/{id}/</code>	CRUD des réservations
<code>/api/salles/{id}/occupation/</code>	taux d'occupation d'une salle

5.1 Tâche 1 — Serializers

Écrivez `SalleSerializer` et `ReservationSerializer` (`ModelSerializer`). Listez les champs **explicitement** (pas de `"__all__"`). Le champ `utilisateur` d'une réservation est en **lecture seule** : il ne doit jamais être fourni par le client.

5.2 Tâche 2 — Validation métier

Dans `ReservationSerializer`, refusez (**HTTP 400**) :

- une réservation dont `fin` n'est pas **strictement postérieure** à `debut` ;
- une réservation qui **chevauche** une autre réservation CONFIRMEE de la **même salle**.

Précisions, qui font partie de la spécification :

- deux créneaux qui se touchent (l'un finit à 10h, l'autre commence à 10h) **ne se chevauchent pas** ;
- des créneaux sur des salles différentes ne se gênent jamais ;
- une réservation ANNULEE ne bloque aucun créneau ;
- **modifier** une réservation existante (PUT ou PATCH, y compris partiel) ne doit pas échouer parce qu'elle chevauche *elle-même*.

5.3 Tâche 3 — ViewSets et router

Un `ModelViewSet` pour les salles et un pour les réservations, branchés sur un `DefaultRouter` dans `salles/urls.py`. À la création d'une réservation, l'utilisateur est **automatiquement** l'utilisateur connecté.

5.4 Tâche 4 — Permissions

- **Réservations** : lecture ouverte à tous ; création réservée aux utilisateurs connectés ; **modification et suppression réservées à l'auteur** de la réservation (écrivez `IsOwnerOrReadOnly` dans `permissions.py`). Un autre utilisateur connecté reçoit **403**.
- **Salles** : lecture ouverte à tous ; création, modification et suppression réservées au **staff**.

5.5 Tâche 5 — Action d'occupation

Ajoutez à `SalleViewSet` une action `@action` accessible en `GET` sur `/api/salles/{id}/occupation/`, avec deux paramètres de requête au format ISO 8601 :

```
GET /api/salles/1/occupation/?debut=2026-11-02T08:00:00Z&fin=2026-11-02T18:00:00Z
```

La réponse JSON contient au minimum la clé `taux_occupation` : un nombre entre 0 et 1, égal à

$$\frac{\text{durée réservée dans la période}}{\text{durée de la période}}$$

Règles de calcul :

- seules les réservations **CONFIRMEE** comptent ;
- une réservation qui **dépasse** de la période ne compte que pour la **partie située dans la période**.

Gestion des erreurs : **400** si `debut` ou `fin` est absent, invalide, ou si `fin` n'est pas postérieure à `debut` ; **404** si la salle n'existe pas.

Pour vérifier votre calcul

Sur l'*Amphi A*, de lundi 8h à lundi 18h, vous devez obtenir 0.4 (4 h réservées sur 10 h).

5.6 Bonus

- **Pagination** de la liste des réservations (`PageNumberPagination`, 10 par page) ;
- **Filtres** sur la liste des réservations : `?salle=<id>` et `?date=AAAA-MM-JJ` (réservations dont le début tombe ce jour-là).

Piège connu

Si vous filtrez en redéfinissant `get_queryset()` sans attribut `queryset` sur le `ViewSet`, le router ne peut plus deviner le nom des routes : ajoutez `basename=...` à `router.register(...)`.

6 Barème (sur 20)

Critère	Pts	Ce qui est évalué	Vérifié par
1. Serializers	2	SalleSerializer correct (1) ; ReservationSerializer avec champs explicites et utilisateur en lecture seule (1)	Tests + code
2. Validation	4	fin > debut (1) ; chevauchement refusé, créneaux qui se touchent et autres salles acceptés (2) ; réservations annulées ignorées et modification non bloquée par elle-même (1)	Tests
3. ViewSets et router	3	deux ModelAndViewSet (1) ; DefaultRouter et routes conformes (1) ; utilisateur = utilisateur connecté à la création (1)	Tests + code
4. Permissions	3	IsOwnerOrReadOnly : non-auteur refusé, auteur accepté, lecture ouverte (2) ; salles : écriture réservée au staff, lecture ouverte (1)	Tests
5. Occupation	3	endpoint fonctionnel avec taux_occupation (1) ; calcul correct : confirmées seulement, parties tronquées à la période (1) ; erreurs 400 et 404 (1)	Tests
6. Collaboration GitHub	3	chaque membre a au moins 2 commits significatifs (2) ; messages de commit explicites (1)	Historique Git
7. README et qualité	2	README complété : membres et rôles, installation, endpoints (1) ; code lisible, sans TODO ni imports inutilisés (1)	Lecture du code
Total	20		
Bonus	+2	pagination (1) ; filtres salle et date (1)	Tests

Précisions sur la notation

- Une partie de la correction est **automatisée** : des tests sont exécutés sur votre dépôt, à partir des URL publiques de l'API. Respectez donc exactement les routes et les comportements décrits plus haut.
- Un commit est **significatif** s'il contient un vrai travail (une fonctionnalité, une correction, de la documentation) : pas un fichier vide, ni un simple renommage.
- Les points de bonus peuvent compenser des points perdus ailleurs ; la note finale est **plafonnée à 20**.

7 Conseils

Répartition suggérée (non obligatoire)

- **Membre A** : serializers et validation (tâches 1 et 2)
- **Membre B** : ViewSets, router et permissions (tâches 3 et 4)
- **Membre C** : action d'occupation, README, puis bonus (tâche 5)

Quelle que soit la répartition, **chacun doit committer**, et chacun doit pouvoir expliquer le code des autres.

Créneau	Repère
0h00 – 0h10	Dépôt GitHub, squelette, installation, serveur lancé
0h10 – 0h40	Serializers et validation
0h40 – 1h10	ViewSets, router, permissions
1h10 – 1h40	Action d'occupation
1h40 – 2h00	Vérifications avec la Browseable API, README, push final

- Testez chaque étape dans la **Browseable API** avant de passer à la suivante.
- Dans `validate()`, en cas de `PATCH`, les champs absents de la requête se lisent sur `self.instance`.
- Pour lire les dates de l'action d'occupation, regardez `django.utils.dateparse.parse_datetime`.

8 Livrables

- Dépôt GitHub partagé avec l'enseignant, contenant le projet complet
- `README.md` complété
- Dernier `push` effectué avant la fin du créneau